

**КАФЕДРА СИСТЕМЫ ОБРАБОТКИ ИНФОРМАЦИИ И УПРАВЛЕНИЯ**

по дисциплине «Технологии машинного обучения»

(И.О. Фамилия)

(И.О. Фамилия)

1

Цель работы

Изучение методов подготовки данных, кросс-валидации и подбора гиперпараметров на примере метода К-ближайших соседей (KNN).

Разделение выборки

```
1 from sklearn.model_selection import train_test_split
2 from sklearn.preprocessing import StandardScaler
3
4 X_train, X_test, y_train, y_test = train_test_split(
5     X, y, test_size=0.3, random_state=42, stratify=y)
6
7 scaler = StandardScaler()
8 X_train_scaled = scaler.fit_transform(X_train)
9 X_test_scaled = scaler.transform(X_test)
```

Базовая модель KNN (K=5)

```
1 from sklearn.neighbors import KNeighborsClassifier
2 from sklearn.metrics import accuracy_score, precision_score
3
4 knn = KNeighborsClassifier(n_neighbors=5)
5 knn.fit(X_train_scaled, y_train)
6
7 y_pred = knn.predict(X_test_scaled)
8 accuracy = accuracy_score(y_test, y_pred)
9 precision = precision_score(y_test, y_pred, average='weighted')
10
11 print(f"Accuracy: {accuracy:.4f}")
12 print(f"Precision: {precision:.4f}")
```

Подбор гиперпараметра K

GridSearchCV

```
1 from sklearn.model_selection import GridSearchCV
2
3 param_grid = {'n_neighbors': list(range(1, 16))}
4 grid = GridSearchCV(KNeighborsClassifier(), param_grid,
5                     cv=5, scoring='accuracy')
6 grid.fit(X_train_scaled, y_train)
7
8 print(f"Best K: {grid.best_params_['n_neighbors']}")
9 print(f"Best CV score: {grid.best_score_:.4f}")
```

RandomizedSearchCV

```
1 from sklearn.model_selection import RandomizedSearchCV
2
3 random_grid = RandomizedSearchCV(KNeighborsClassifier(),
4                                   param_grid, cv=5,
5                                   n_iter=10, random_state=42)
6 random_grid.fit(X_train_scaled, y_train)
```

Кросс-валидация

```
1 from sklearn.model_selection import cross_val_score, StratifiedKFold
2
3 # K-Fold
4 kf = KFold(n_splits=5, shuffle=True, random_state=42)
5 scores_kf = cross_val_score(knn, X_train_scaled, y_train,
6                             cv=kf, scoring='accuracy')
7
8 # Stratified K-Fold сохраняет пропорции классов
9 skf = StratifiedKFold(n_splits=5, shuffle=True, random_state=42)
10 scores_skf = cross_val_score(knn, X_train_scaled, y_train,
11                              cv=skf, scoring='accuracy')
12
13 print(f"K-Fold scores: {scores_kf.mean():.4f} ± {scores_kf.std():.4f}")
14 print(f"Stratified K-Fold scores: {scores_skf.mean():.4f} ± {scores_skf.std():.4f}")
```

Результаты

- **Базовая модель (K=5):** Accuracy = 0.9778
- **GridSearchCV:** Best K = 3, CV score = 0.9815
- **RandomizedSearchCV:** Best K = 4, score ≈ 0.9800
- **Улучшение:** +0.4% на тестовой выборке

Stratified K-Fold обеспечил более стабильные результаты благодаря сохранению пропорций классов в каждой fold'е.

Выводы

1. GridSearchCV надежнее для полного перебора параметров
2. Stratified K-Fold предпочтительна для несбалансированных данных
3. Оптимальное K зависит от размера и структуры данных
4. Нормализация данных (StandardScaler) критична для KNN